To validate the Mongoose ObjectId:

1. The recommended way:

```
const mongoose = require('mongoose');

const id1 = '62261a65d66c6be0a63c051f'; // Valid hex string
const id2 = 'zzzzzzzzzzz';              // 12-character non-hex string

console.log(mongoose.isObjectIdOrHexString(id1)); // true
console.log(mongoose.isObjectIdOrHexString(id2)); // false
```

---

2. The standard choice:

```
const mongoose = require('mongoose');
console.log(mongoose.isValidObjectId('62261a65d66c6be0a63c051f')); // true
```

Note: This method (along with its lower-level counterpart mongoose.Types.ObjectId.isValid()) defers to the BSON library. Because MongoDB treats any 12-byte raw string as a valid ID format, a 12-character non-hex string (like "zzzzzzzzzzz") will falsely return true.

==========

To define a new Mongoose schema:

```
const mongoose = require("mongoose");

const productSchema = new mongoose.Schema({
    name: {
        type: String,
        required: true,
        minlength: 3,
        maxlength: 150,
    },

    description: {
        type: String,
        required: true,
    },

    richDescription: {
        type: String,
        default: '',
    },

    image: {
        type: String,
        default: '',
```

```
},

images: {
    type: [ String ],
    default: ''
},

brand: {
    type: String,
    default: ''
},

price: {
    type: Number,
    required: true,
    min: 1,
},

category: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Category',
    required: true,
},

numberInStock: {
    type: Number,
    required: true,
    min: 0,
    max: 15000,
},

rating: {
    type: Number,
    default: 0,
},

numReviews: {
    type: Number,
    default: 0,
},

isFeatured: {
    type: Boolean,
    default: false,
},
```

```
    dateCreated: {
        type: Date,
        default: Date.now,
    },
});
```

==========

To export the previous Mongoose schema as a Model:

```
const ProductModel = mongoose.model('Product', productSchema)

module.exports.Product = ProductModel
module.exports.productSchema = productSchema
```

==========

To get the nested relation of a specific model:

```
router.get('/:id', [objectId, auth, isAdmin], async (req, res) => {
    let order = await Order
        .findOne({ _id: req.params.id })
        .populate('user', '-passwordHash')
        .populate({
            path: 'orderItems',
            populate: {
                path: 'product',
                populate: 'category',
            }
        })

    if(!order) {
        return res.status(404).json({
            status: false,
            msg: 'NO Data Found For Your Id'
        }).end();
    }

    return res.status(200).json({
        status: true,
        msg: 'Successfully Getting Order',
        'data': order,
    }).end();
});
```

==========

To define nested relations between models:

- For arrays:

```
orderItems: [{
        type: mongoose.Schema.Types.ObjectId,
        ref: 'OrderItem',
        required: true,
    }],
```

- For one item:

```
user: {
        type: mongoose.Schema.Types.ObjectId,
        ref: 'User',
        required: true,
    },
```

==========

To use the Mongoose methods to check the data:

```
// Hash password before saving to database
userSchema.pre('save', async function () {
    // Only hash the password if it has been modified (or is new)
    if (!this.isModified('password')) {
        return;
    }

    try {
        // Generate salt and hash password
        const salt = await bcrypt.genSalt(10);
        this.password = await bcrypt.hash(this.password, salt);
    } catch (error) {
        throw error
    }
});
```

==========

```
// Method to compare entered password with hashed password in database
userSchema.methods.matchPassword = async function (enteredPassword) {
    return await bcrypt.compare(enteredPassword, this.password);
};
```

==========

Adding an index to the model:

```
// Optional: Add a compound index if you often query tasks by user and creation
date
taskSchema.index({ user_id: 1, createdAt: -1 });
```

==========

To add validation messages to the created schema:

```
const taskSchema = new mongoose.Schema(
    {
        // task name: required, trimmed, with a max length
        name: {
            type: String,
            required: [true, 'Task name is required'],
            trim: true,
            maxlength: [100, 'Task name cannot be more than 100 characters'],
        },
        // task description: optional, but trimmed
        description: {
            type: String,
            trim: true,
            maxlength: [500, 'Description cannot be more than 500 characters'],
        },
        // user_id: reference to the User model, required
        user_id: {
            type: mongoose.Schema.Types.ObjectId,
            required: [true, 'User ID is required'],
            index: true, // Indexing for faster queries by user
        },
    },
    {
        timestamps: true, // Automatically adds createdAt and updatedAt fields
    }
);
```

==========